

Section 9.3 - Convnet architecture patterns

December 2, 2025

1 Convnet architecture patterns

- Model architecture = sum of choices: layers & their configurations, arrangements
- Define the hypothesis space: possible functions that gradient descent can search over
- Model architecture: more an art than a science
- Good assumptions about what to look for → effective learning from data
- Good model architecture → reduce size of search space
- No clear explanation of what works & what doesn't work
- Like science in engineering discipline → best practices
- Modern DL: product of co-evolution process of hardware, software and algorithms
- A different path may require multi-year re-engineering of the new ecosystem

1.1 Modularity, hierarchy & reuse (MHR)

- 1) Structure in modules
- 2) Organize modules in hierarchy (pyramid-like structures)
- 3) Reuse modules as appropriate
 - Deep hierarchy: encourage feature reuse, but vanishing gradients
 - Ablation study: systematic try to remove parts of a system
 - Could there be a simpler explanation?
 - Is this added complexity really necessary? Why?
 - DL architecture: more evolved than designed

1.2 Residual connections

- Backpropagation - like game of telephone
- Chain too deep → noise overwhelms gradient: the vanishing gradient problem
- Cumulative error - the end may bear little of its origin
- Add input of a layer or block back to its output
- Linearly project residual to output shape
- Use strides to match any downsampling

```
[1]: # Residual connection
from tensorflow import keras
from tensorflow.keras import layers

inputs = keras.Input(shape=(32, 32, 3))
```

```
x = layers.Conv2D(32, 3, activation="relu")(inputs)
residual = x
x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)
residual = layers.Conv2D(64, 1)(residual)
x = layers.add([x, residual])
```

Layer with a max pooling layer

```
[2]: inputs = keras.Input(shape=(32, 32, 3))
x = layers.Conv2D(32, 3, activation="relu")(inputs)
residual = x
x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)
x = layers.MaxPooling2D(2, padding="same")(x)
residual = layers.Conv2D(64, 1, strides=2)(residual)
x = layers.add([x, residual])
```

An example

```
[3]: inputs = keras.Input(shape=(32, 32, 3))
x = layers.Rescaling(1./255)(inputs)

def residual_block(x, filters, pooling=False):
    residual = x
    x = layers.Conv2D(filters, 3, activation="relu", padding="same")(x)
    x = layers.Conv2D(filters, 3, activation="relu", padding="same")(x)
    if pooling:
        x = layers.MaxPooling2D(2, padding="same")(x)
        residual = layers.Conv2D(filters, 1, strides=2)(residual)
    elif filters != residual.shape[-1]:
        residual = layers.Conv2D(filters, 1)(residual)
    x = layers.add([x, residual])
    return x

x = residual_block(x, filters=32, pooling=True)
x = residual_block(x, filters=64, pooling=True)
x = residual_block(x, filters=128, pooling=False)

x = layers.GlobalAveragePooling2D()(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs=inputs, outputs=outputs)
model.summary()
```

Model: "model"

Layer (type)	Output Shape	Param #	Connected to
input_3 (InputLayer)	[(None, 32, 32, 3)]	0	[]

rescaling (Rescaling) ['input_3[0][0]']	(None, 32, 32, 3)	0	
conv2d_6 (Conv2D) ['rescaling[0][0]']	(None, 32, 32, 32)	896	
conv2d_7 (Conv2D) ['conv2d_6[0][0]']	(None, 32, 32, 32)	9248	
max_pooling2d_1 (MaxPooling2D) ['conv2d_7[0][0]']	(None, 16, 16, 32)	0	
conv2d_8 (Conv2D) ['rescaling[0][0]']	(None, 16, 16, 32)	128	
add_2 (Add) ['max_pooling2d_1[0][0]', 'conv2d_8[0][0]']	(None, 16, 16, 32)	0	
conv2d_9 (Conv2D)	(None, 16, 16, 64)	18496	['add_2[0][0]']
conv2d_10 (Conv2D) ['conv2d_9[0][0]']	(None, 16, 16, 64)	36928	
max_pooling2d_2 (MaxPooling2D) ['conv2d_10[0][0]']	(None, 8, 8, 64)	0	
conv2d_11 (Conv2D)	(None, 8, 8, 64)	2112	['add_2[0][0]']
add_3 (Add) ['max_pooling2d_2[0][0]', 'conv2d_11[0][0]']	(None, 8, 8, 64)	0	
conv2d_12 (Conv2D)	(None, 8, 8, 128)	73856	['add_3[0][0]']
conv2d_13 (Conv2D) ['conv2d_12[0][0]']	(None, 8, 8, 128)	147584	
conv2d_14 (Conv2D)	(None, 8, 8, 128)	8320	['add_3[0][0]']
add_4 (Add) ['conv2d_13[0][0]', 'conv2d_14[0][0]']	(None, 8, 8, 128)	0	
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	0	['add_4[0][0]']

```
dense (Dense)                (None, 1)                129
['global_average_pooling2d[0][0]'
```

```
]
```

```
=====
Total params: 297,697
Trainable params: 297,697
Non-trainable params: 0
-----
```

1.3 Batch normalization

- Make samples uniform/similar
- Adaptively normalize data during training
- There are hypotheses, but not certitudes! Only best practices
- Know the how, but do not know the why!
- Normalize outputs -> No need of bias vector
- Fine-tuning: freeze BatchNormalization layers (recommended)

1.4 Depthwise separable convolutions

- Convolution on each channel, then mix them with 1x1 kernel
- Drop-in replacement for Conv2D
- Leaner - fewer floating-point operations (smaller models)?
- Assumption: patterns are not tied to specific locations
- Spatial locations in intermediate activations are highly correlated, but channels are highly independent
- SeparableConv2D is not ready at NVIDIA!

1.5 A mini Xception-like model

```
[4]: import os
import tensorflow as tf
import logging

# Suppress TensorFlow INFO and WARNING messages
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
logging.getLogger('tensorflow').setLevel(logging.ERROR)
```

Dataset

```
[5]: from pathlib import Path
from tensorflow.keras.utils import image_dataset_from_directory

new_base_dir = Path("cats_vs_dogs_small")

train_dataset = image_dataset_from_directory(
```

```

new_base_dir / "train",
image_size=(180, 180),
batch_size=32)
validation_dataset = image_dataset_from_directory(
    new_base_dir / "validation",
    image_size=(180, 180),
    batch_size=32)
test_dataset = image_dataset_from_directory(
    new_base_dir / "test",
    image_size=(180, 180),
    batch_size=32)

```

Found 2000 files belonging to 2 classes.
 Found 1000 files belonging to 2 classes.
 Found 2000 files belonging to 2 classes.

Model architecture

```

[6]: from tensorflow import keras
    from tensorflow.keras import layers

data_augmentation = keras.Sequential(
    [
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(0.1),
        layers.RandomZoom(0.2),
    ]
)

inputs = keras.Input(shape=(180, 180, 3))
x = data_augmentation(inputs)

x = layers.Rescaling(1./255)(x)
x = layers.Conv2D(filters=32, kernel_size=5, use_bias=False)(x)

for size in [32, 64, 128, 256, 512]:
    residual = x

    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.SeparableConv2D(size, 3, padding="same", use_bias=False)(x)

    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.SeparableConv2D(size, 3, padding="same", use_bias=False)(x)

    x = layers.MaxPooling2D(3, strides=2, padding="same")(x)

```

```

        residual = layers.Conv2D(
            size, 1, strides=2, padding="same", use_bias=False)(residual)
        x = layers.add([x, residual])

x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)

model = keras.Model(inputs=inputs, outputs=outputs)

model.summary()

```

Model: "model_1"

Layer (type)	Output Shape	Param #	Connected to
=====			
input_4 (InputLayer)	[(None, 180, 180, 3) 0)]		[]
sequential (Sequential) ['input_4[0][0]']	(None, 180, 180, 3) 0		
rescaling_1 (Rescaling) ['sequential[0][0]']	(None, 180, 180, 3) 0		
conv2d_15 (Conv2D) ['rescaling_1[0][0]']	(None, 176, 176, 32) 2400)		
batch_normalization (BatchNorm ['conv2d_15[0][0]'] alization)	(None, 176, 176, 32) 128)		
activation (Activation) ['batch_normalization[0][0]']	(None, 176, 176, 32) 0)		
separable_conv2d (SeparableCon ['activation[0][0]'] v2D)	(None, 176, 176, 32) 1312)		
batch_normalization_1 (BatchNo ['separable_conv2d[0][0]'] rmalization)	(None, 176, 176, 32) 128)		
activation_1 (Activation)	(None, 176, 176, 32) 0		

```

['batch_normalization_1[0][0]']
    )

separable_conv2d_1 (SeparableC (None, 176, 176, 32 1312
['activation_1[0][0]']
    onv2D)
    )

max_pooling2d_3 (MaxPooling2D) (None, 88, 88, 32) 0
['separable_conv2d_1[0][0]']

conv2d_16 (Conv2D) (None, 88, 88, 32) 1024
['conv2d_15[0][0]']

add_5 (Add) (None, 88, 88, 32) 0
['max_pooling2d_3[0][0]',
'conv2d_16[0][0]']

batch_normalization_2 (BatchNo (None, 88, 88, 32) 128 ['add_5[0][0]']
    rmalization)

activation_2 (Activation) (None, 88, 88, 32) 0
['batch_normalization_2[0][0]']

separable_conv2d_2 (SeparableC (None, 88, 88, 64) 2336
['activation_2[0][0]']
    onv2D)

batch_normalization_3 (BatchNo (None, 88, 88, 64) 256
['separable_conv2d_2[0][0]']
    rmalization)

activation_3 (Activation) (None, 88, 88, 64) 0
['batch_normalization_3[0][0]']

separable_conv2d_3 (SeparableC (None, 88, 88, 64) 4672
['activation_3[0][0]']
    onv2D)

max_pooling2d_4 (MaxPooling2D) (None, 44, 44, 64) 0
['separable_conv2d_3[0][0]']

conv2d_17 (Conv2D) (None, 44, 44, 64) 2048 ['add_5[0][0]']

add_6 (Add) (None, 44, 44, 64) 0
['max_pooling2d_4[0][0]',
'conv2d_17[0][0]']

batch_normalization_4 (BatchNo (None, 44, 44, 64) 256 ['add_6[0][0]']
    rmalization)

```

```

rmalization)

activation_4 (Activation)      (None, 44, 44, 64)    0
['batch_normalization_4[0][0]']

separable_conv2d_4 (SeparableC (None, 44, 44, 128)  8768
['activation_4[0][0]']
onv2D)

batch_normalization_5 (BatchNo (None, 44, 44, 128)  512
['separable_conv2d_4[0][0]']
rmalization)

activation_5 (Activation)      (None, 44, 44, 128)    0
['batch_normalization_5[0][0]']

separable_conv2d_5 (SeparableC (None, 44, 44, 128)  17536
['activation_5[0][0]']
onv2D)

max_pooling2d_5 (MaxPooling2D) (None, 22, 22, 128)    0
['separable_conv2d_5[0][0]']

conv2d_18 (Conv2D)             (None, 22, 22, 128)  8192      ['add_6[0][0]']

add_7 (Add)                    (None, 22, 22, 128)    0
['max_pooling2d_5[0][0]',
'conv2d_18[0][0]']

batch_normalization_6 (BatchNo (None, 22, 22, 128)  512      ['add_7[0][0]']
rmalization)

activation_6 (Activation)      (None, 22, 22, 128)    0
['batch_normalization_6[0][0]']

separable_conv2d_6 (SeparableC (None, 22, 22, 256)  33920
['activation_6[0][0]']
onv2D)

batch_normalization_7 (BatchNo (None, 22, 22, 256)  1024
['separable_conv2d_6[0][0]']
rmalization)

activation_7 (Activation)      (None, 22, 22, 256)    0
['batch_normalization_7[0][0]']

separable_conv2d_7 (SeparableC (None, 22, 22, 256)  67840

```



```

['activation_7[0][0]']
conv2D)

max_pooling2d_6 (MaxPooling2D) (None, 11, 11, 256) 0
['separable_conv2d_7[0][0]']

conv2d_19 (Conv2D) (None, 11, 11, 256) 32768 ['add_7[0][0]']

add_8 (Add) (None, 11, 11, 256) 0
['max_pooling2d_6[0][0]',
'conv2d_19[0][0]']

batch_normalization_8 (BatchNormaliz (None, 11, 11, 256) 1024 ['add_8[0][0]']
ation)

activation_8 (Activation) (None, 11, 11, 256) 0
['batch_normalization_8[0][0]']

separable_conv2d_8 (SeparableConv2D) (None, 11, 11, 512) 133376
['activation_8[0][0]']
conv2D)

batch_normalization_9 (BatchNormaliz (None, 11, 11, 512) 2048
ation)
['separable_conv2d_8[0][0]']

activation_9 (Activation) (None, 11, 11, 512) 0
['batch_normalization_9[0][0]']

separable_conv2d_9 (SeparableConv2D) (None, 11, 11, 512) 266752
['activation_9[0][0]']
conv2D)

max_pooling2d_7 (MaxPooling2D) (None, 6, 6, 512) 0
['separable_conv2d_9[0][0]']

conv2d_20 (Conv2D) (None, 6, 6, 512) 131072 ['add_8[0][0]']

add_9 (Add) (None, 6, 6, 512) 0
['max_pooling2d_7[0][0]',
'conv2d_20[0][0]']

global_average_pooling2d_1 (GlobalAveragePooling2D) (None, 512) 0 ['add_9[0][0]']

dropout (Dropout) (None, 512) 0
['global_average_pooling2d_1[0][0]']
]']

```

```
dense_1 (Dense)                (None, 1)                513
['dropout[0][0]']
```

```
=====
Total params: 721,857
Trainable params: 718,849
Non-trainable params: 3,008
-----
```

Training the model

```
[7]: model.compile(loss="binary_crossentropy",
                    optimizer="rmsprop",
                    metrics=["accuracy"])

callbacks = [
    keras.callbacks.ModelCheckpoint(
        filepath="mini_Xception_like.h5",
        save_best_only=True,
        monitor="val_loss")
]

history = model.fit(
    train_dataset,
    epochs=100,
    validation_data=validation_dataset,
    callbacks=callbacks)
```

Epoch 1/100

```
63/63 [=====] - 16s 157ms/step - loss: 0.7135 -
accuracy: 0.5550 - val_loss: 0.6931 - val_accuracy: 0.5160
```

Epoch 2/100

```
63/63 [=====] - 9s 136ms/step - loss: 0.6600 -
accuracy: 0.5895 - val_loss: 0.6982 - val_accuracy: 0.5000
```

Epoch 3/100

```
63/63 [=====] - 9s 137ms/step - loss: 0.6471 -
accuracy: 0.6165 - val_loss: 0.6930 - val_accuracy: 0.5010
```

Epoch 4/100

```
63/63 [=====] - 9s 136ms/step - loss: 0.6265 -
accuracy: 0.6515 - val_loss: 0.6960 - val_accuracy: 0.5000
```

Epoch 5/100

```
63/63 [=====] - 9s 137ms/step - loss: 0.5971 -
accuracy: 0.6855 - val_loss: 0.7179 - val_accuracy: 0.5000
```

Epoch 6/100

```
63/63 [=====] - 9s 136ms/step - loss: 0.5967 -
accuracy: 0.6835 - val_loss: 0.7183 - val_accuracy: 0.5000
```

Epoch 7/100
63/63 [=====] - 9s 137ms/step - loss: 0.5652 - accuracy: 0.7165 - val_loss: 0.7694 - val_accuracy: 0.5020

Epoch 8/100
63/63 [=====] - 9s 137ms/step - loss: 0.5426 - accuracy: 0.7210 - val_loss: 0.7531 - val_accuracy: 0.5210

Epoch 9/100
63/63 [=====] - 9s 137ms/step - loss: 0.5415 - accuracy: 0.7360 - val_loss: 0.7648 - val_accuracy: 0.5550

Epoch 10/100
63/63 [=====] - 9s 138ms/step - loss: 0.5330 - accuracy: 0.7370 - val_loss: 0.6175 - val_accuracy: 0.6930

Epoch 11/100
63/63 [=====] - 9s 136ms/step - loss: 0.4957 - accuracy: 0.7510 - val_loss: 0.8368 - val_accuracy: 0.5960

Epoch 12/100
63/63 [=====] - 9s 137ms/step - loss: 0.4988 - accuracy: 0.7485 - val_loss: 0.7521 - val_accuracy: 0.6140

Epoch 13/100
63/63 [=====] - 9s 139ms/step - loss: 0.4798 - accuracy: 0.7825 - val_loss: 0.5533 - val_accuracy: 0.7410

Epoch 14/100
63/63 [=====] - 9s 137ms/step - loss: 0.4956 - accuracy: 0.7665 - val_loss: 1.2048 - val_accuracy: 0.5390

Epoch 15/100
63/63 [=====] - 9s 137ms/step - loss: 0.4600 - accuracy: 0.7830 - val_loss: 0.6227 - val_accuracy: 0.7080

Epoch 16/100
63/63 [=====] - 9s 135ms/step - loss: 0.4629 - accuracy: 0.7945 - val_loss: 0.7230 - val_accuracy: 0.6150

Epoch 17/100
63/63 [=====] - 9s 141ms/step - loss: 0.4508 - accuracy: 0.7895 - val_loss: 0.7793 - val_accuracy: 0.6330

Epoch 18/100
63/63 [=====] - 9s 144ms/step - loss: 0.4364 - accuracy: 0.7895 - val_loss: 0.6134 - val_accuracy: 0.6570

Epoch 19/100
63/63 [=====] - 10s 156ms/step - loss: 0.4194 - accuracy: 0.8120 - val_loss: 0.6459 - val_accuracy: 0.6990

Epoch 20/100
63/63 [=====] - 10s 152ms/step - loss: 0.4036 - accuracy: 0.8170 - val_loss: 0.6383 - val_accuracy: 0.7440

Epoch 21/100
63/63 [=====] - 9s 144ms/step - loss: 0.3900 - accuracy: 0.8195 - val_loss: 0.6159 - val_accuracy: 0.7400

Epoch 22/100
63/63 [=====] - 9s 141ms/step - loss: 0.4023 - accuracy: 0.8175 - val_loss: 0.5463 - val_accuracy: 0.7430

Epoch 23/100
63/63 [=====] - 9s 141ms/step - loss: 0.3863 -
accuracy: 0.8235 - val_loss: 1.4223 - val_accuracy: 0.6290
Epoch 24/100
63/63 [=====] - 9s 141ms/step - loss: 0.3666 -
accuracy: 0.8390 - val_loss: 0.5316 - val_accuracy: 0.7450
Epoch 25/100
63/63 [=====] - 9s 142ms/step - loss: 0.3591 -
accuracy: 0.8415 - val_loss: 0.4562 - val_accuracy: 0.7990
Epoch 26/100
63/63 [=====] - 9s 140ms/step - loss: 0.3465 -
accuracy: 0.8500 - val_loss: 1.2517 - val_accuracy: 0.6130
Epoch 27/100
63/63 [=====] - 9s 140ms/step - loss: 0.3349 -
accuracy: 0.8545 - val_loss: 0.7221 - val_accuracy: 0.7060
Epoch 28/100
63/63 [=====] - 9s 140ms/step - loss: 0.3274 -
accuracy: 0.8510 - val_loss: 0.8686 - val_accuracy: 0.6960
Epoch 29/100
63/63 [=====] - 9s 138ms/step - loss: 0.3039 -
accuracy: 0.8665 - val_loss: 1.3928 - val_accuracy: 0.6250
Epoch 30/100
63/63 [=====] - 9s 138ms/step - loss: 0.2880 -
accuracy: 0.8760 - val_loss: 0.5363 - val_accuracy: 0.7890
Epoch 31/100
63/63 [=====] - 9s 139ms/step - loss: 0.2823 -
accuracy: 0.8720 - val_loss: 0.5558 - val_accuracy: 0.7770
Epoch 32/100
63/63 [=====] - 9s 140ms/step - loss: 0.2600 -
accuracy: 0.8960 - val_loss: 0.5012 - val_accuracy: 0.7800
Epoch 33/100
63/63 [=====] - 9s 141ms/step - loss: 0.2918 -
accuracy: 0.8755 - val_loss: 0.5673 - val_accuracy: 0.7930
Epoch 34/100
63/63 [=====] - 9s 140ms/step - loss: 0.2732 -
accuracy: 0.8770 - val_loss: 0.5692 - val_accuracy: 0.7990
Epoch 35/100
63/63 [=====] - 9s 140ms/step - loss: 0.2534 -
accuracy: 0.8945 - val_loss: 0.5601 - val_accuracy: 0.7800
Epoch 36/100
63/63 [=====] - 9s 144ms/step - loss: 0.2641 -
accuracy: 0.8930 - val_loss: 0.5141 - val_accuracy: 0.7910
Epoch 37/100
63/63 [=====] - 9s 141ms/step - loss: 0.2631 -
accuracy: 0.8950 - val_loss: 0.4835 - val_accuracy: 0.8060
Epoch 38/100
63/63 [=====] - 9s 141ms/step - loss: 0.2429 -
accuracy: 0.9025 - val_loss: 0.5504 - val_accuracy: 0.7480

Epoch 39/100
63/63 [=====] - 9s 141ms/step - loss: 0.2465 - accuracy: 0.9010 - val_loss: 0.4957 - val_accuracy: 0.8130
Epoch 40/100
63/63 [=====] - 9s 143ms/step - loss: 0.2213 - accuracy: 0.9085 - val_loss: 0.6685 - val_accuracy: 0.7640
Epoch 41/100
63/63 [=====] - 9s 140ms/step - loss: 0.2244 - accuracy: 0.9140 - val_loss: 0.5641 - val_accuracy: 0.7930
Epoch 42/100
63/63 [=====] - 9s 141ms/step - loss: 0.2249 - accuracy: 0.9080 - val_loss: 0.5616 - val_accuracy: 0.7720
Epoch 43/100
63/63 [=====] - 9s 141ms/step - loss: 0.2122 - accuracy: 0.9105 - val_loss: 0.5759 - val_accuracy: 0.7980
Epoch 44/100
63/63 [=====] - 9s 142ms/step - loss: 0.1908 - accuracy: 0.9225 - val_loss: 0.5837 - val_accuracy: 0.8030
Epoch 45/100
63/63 [=====] - 9s 142ms/step - loss: 0.2019 - accuracy: 0.9120 - val_loss: 0.4447 - val_accuracy: 0.8160
Epoch 46/100
63/63 [=====] - 9s 140ms/step - loss: 0.2015 - accuracy: 0.9200 - val_loss: 1.7470 - val_accuracy: 0.5940
Epoch 47/100
63/63 [=====] - 9s 141ms/step - loss: 0.2008 - accuracy: 0.9155 - val_loss: 1.2003 - val_accuracy: 0.6730
Epoch 48/100
63/63 [=====] - 9s 144ms/step - loss: 0.1617 - accuracy: 0.9360 - val_loss: 1.2724 - val_accuracy: 0.7090
Epoch 49/100
63/63 [=====] - 9s 143ms/step - loss: 0.2009 - accuracy: 0.9180 - val_loss: 0.6561 - val_accuracy: 0.7830
Epoch 50/100
63/63 [=====] - 9s 144ms/step - loss: 0.1816 - accuracy: 0.9255 - val_loss: 0.4272 - val_accuracy: 0.8490
Epoch 51/100
63/63 [=====] - 9s 140ms/step - loss: 0.1895 - accuracy: 0.9210 - val_loss: 0.6708 - val_accuracy: 0.8050
Epoch 52/100
63/63 [=====] - 9s 140ms/step - loss: 0.1708 - accuracy: 0.9265 - val_loss: 0.5694 - val_accuracy: 0.8090
Epoch 53/100
63/63 [=====] - 9s 142ms/step - loss: 0.1755 - accuracy: 0.9325 - val_loss: 0.4515 - val_accuracy: 0.8320
Epoch 54/100
63/63 [=====] - 9s 144ms/step - loss: 0.1563 - accuracy: 0.9375 - val_loss: 0.4117 - val_accuracy: 0.8500

Epoch 55/100
63/63 [=====] - 9s 139ms/step - loss: 0.1521 -
accuracy: 0.9405 - val_loss: 0.5020 - val_accuracy: 0.8170
Epoch 56/100
63/63 [=====] - 9s 141ms/step - loss: 0.1589 -
accuracy: 0.9370 - val_loss: 1.7449 - val_accuracy: 0.6690
Epoch 57/100
63/63 [=====] - 9s 143ms/step - loss: 0.1507 -
accuracy: 0.9435 - val_loss: 0.4709 - val_accuracy: 0.8480
Epoch 58/100
63/63 [=====] - 9s 144ms/step - loss: 0.1499 -
accuracy: 0.9405 - val_loss: 0.3725 - val_accuracy: 0.8680
Epoch 59/100
63/63 [=====] - 9s 142ms/step - loss: 0.1661 -
accuracy: 0.9335 - val_loss: 0.4023 - val_accuracy: 0.8470
Epoch 60/100
63/63 [=====] - 9s 141ms/step - loss: 0.1460 -
accuracy: 0.9435 - val_loss: 0.7099 - val_accuracy: 0.8080
Epoch 61/100
63/63 [=====] - 9s 141ms/step - loss: 0.1399 -
accuracy: 0.9445 - val_loss: 1.5731 - val_accuracy: 0.7170
Epoch 62/100
63/63 [=====] - 9s 142ms/step - loss: 0.1279 -
accuracy: 0.9550 - val_loss: 0.5145 - val_accuracy: 0.8270
Epoch 63/100
63/63 [=====] - 9s 144ms/step - loss: 0.1290 -
accuracy: 0.9485 - val_loss: 0.4437 - val_accuracy: 0.8460
Epoch 64/100
63/63 [=====] - 9s 143ms/step - loss: 0.1293 -
accuracy: 0.9495 - val_loss: 1.0258 - val_accuracy: 0.7550
Epoch 65/100
63/63 [=====] - 9s 143ms/step - loss: 0.1233 -
accuracy: 0.9500 - val_loss: 0.5725 - val_accuracy: 0.8270
Epoch 66/100
63/63 [=====] - 9s 143ms/step - loss: 0.1308 -
accuracy: 0.9510 - val_loss: 0.4544 - val_accuracy: 0.8650
Epoch 67/100
63/63 [=====] - 9s 142ms/step - loss: 0.1206 -
accuracy: 0.9530 - val_loss: 0.4530 - val_accuracy: 0.8440
Epoch 68/100
63/63 [=====] - 9s 141ms/step - loss: 0.1346 -
accuracy: 0.9465 - val_loss: 0.5593 - val_accuracy: 0.8460
Epoch 69/100
63/63 [=====] - 9s 141ms/step - loss: 0.1291 -
accuracy: 0.9485 - val_loss: 0.5413 - val_accuracy: 0.8500
Epoch 70/100
63/63 [=====] - 9s 142ms/step - loss: 0.1332 -

accuracy: 0.9505 - val_loss: 0.5691 - val_accuracy: 0.8240
Epoch 71/100
63/63 [=====] - 9s 142ms/step - loss: 0.1146 -
accuracy: 0.9535 - val_loss: 0.4129 - val_accuracy: 0.8720
Epoch 72/100
63/63 [=====] - 9s 142ms/step - loss: 0.1059 -
accuracy: 0.9565 - val_loss: 0.5410 - val_accuracy: 0.8620
Epoch 73/100
63/63 [=====] - 9s 142ms/step - loss: 0.1172 -
accuracy: 0.9570 - val_loss: 0.4467 - val_accuracy: 0.8670
Epoch 74/100
63/63 [=====] - 9s 141ms/step - loss: 0.1288 -
accuracy: 0.9505 - val_loss: 1.1480 - val_accuracy: 0.7770
Epoch 75/100
63/63 [=====] - 9s 140ms/step - loss: 0.1179 -
accuracy: 0.9565 - val_loss: 0.4621 - val_accuracy: 0.8580
Epoch 76/100
63/63 [=====] - 9s 142ms/step - loss: 0.1041 -
accuracy: 0.9610 - val_loss: 0.5266 - val_accuracy: 0.8280
Epoch 77/100
63/63 [=====] - 9s 141ms/step - loss: 0.0962 -
accuracy: 0.9630 - val_loss: 0.5473 - val_accuracy: 0.8510
Epoch 78/100
63/63 [=====] - 9s 142ms/step - loss: 0.1035 -
accuracy: 0.9600 - val_loss: 0.5659 - val_accuracy: 0.8430
Epoch 79/100
63/63 [=====] - 9s 142ms/step - loss: 0.0960 -
accuracy: 0.9640 - val_loss: 0.4563 - val_accuracy: 0.8630
Epoch 80/100
63/63 [=====] - 9s 143ms/step - loss: 0.1188 -
accuracy: 0.9565 - val_loss: 0.7396 - val_accuracy: 0.8070
Epoch 81/100
63/63 [=====] - 9s 141ms/step - loss: 0.0975 -
accuracy: 0.9625 - val_loss: 0.4739 - val_accuracy: 0.8690
Epoch 82/100
63/63 [=====] - 9s 141ms/step - loss: 0.0897 -
accuracy: 0.9630 - val_loss: 2.1338 - val_accuracy: 0.6480
Epoch 83/100
63/63 [=====] - 9s 143ms/step - loss: 0.1189 -
accuracy: 0.9545 - val_loss: 0.6367 - val_accuracy: 0.8330
Epoch 84/100
63/63 [=====] - 9s 141ms/step - loss: 0.1164 -
accuracy: 0.9580 - val_loss: 0.4903 - val_accuracy: 0.8480
Epoch 85/100
63/63 [=====] - 9s 142ms/step - loss: 0.1132 -
accuracy: 0.9615 - val_loss: 1.0269 - val_accuracy: 0.7860
Epoch 86/100
63/63 [=====] - 9s 143ms/step - loss: 0.0885 -

```

accuracy: 0.9660 - val_loss: 1.3025 - val_accuracy: 0.7630
Epoch 87/100
63/63 [=====] - 9s 143ms/step - loss: 0.0882 -
accuracy: 0.9680 - val_loss: 1.1396 - val_accuracy: 0.7580
Epoch 88/100
63/63 [=====] - 9s 141ms/step - loss: 0.1070 -
accuracy: 0.9550 - val_loss: 0.4404 - val_accuracy: 0.8500
Epoch 89/100
63/63 [=====] - 9s 143ms/step - loss: 0.1022 -
accuracy: 0.9600 - val_loss: 0.4732 - val_accuracy: 0.8680
Epoch 90/100
63/63 [=====] - 9s 142ms/step - loss: 0.0918 -
accuracy: 0.9655 - val_loss: 0.5976 - val_accuracy: 0.8480
Epoch 91/100
63/63 [=====] - 9s 142ms/step - loss: 0.0802 -
accuracy: 0.9670 - val_loss: 0.5104 - val_accuracy: 0.8730
Epoch 92/100
63/63 [=====] - 9s 143ms/step - loss: 0.0953 -
accuracy: 0.9680 - val_loss: 0.4218 - val_accuracy: 0.8710
Epoch 93/100
63/63 [=====] - 9s 142ms/step - loss: 0.0834 -
accuracy: 0.9725 - val_loss: 0.8854 - val_accuracy: 0.7830
Epoch 94/100
63/63 [=====] - 9s 141ms/step - loss: 0.0787 -
accuracy: 0.9680 - val_loss: 0.6531 - val_accuracy: 0.8430
Epoch 95/100
63/63 [=====] - 9s 142ms/step - loss: 0.1033 -
accuracy: 0.9650 - val_loss: 0.6216 - val_accuracy: 0.8460
Epoch 96/100
63/63 [=====] - 9s 143ms/step - loss: 0.0808 -
accuracy: 0.9705 - val_loss: 0.8581 - val_accuracy: 0.7950
Epoch 97/100
63/63 [=====] - 9s 144ms/step - loss: 0.0820 -
accuracy: 0.9690 - val_loss: 0.7742 - val_accuracy: 0.8460
Epoch 98/100
63/63 [=====] - 9s 144ms/step - loss: 0.0808 -
accuracy: 0.9700 - val_loss: 0.8013 - val_accuracy: 0.8130
Epoch 99/100
63/63 [=====] - 9s 142ms/step - loss: 0.0968 -
accuracy: 0.9620 - val_loss: 0.5551 - val_accuracy: 0.8400
Epoch 100/100
63/63 [=====] - 9s 141ms/step - loss: 0.0744 -
accuracy: 0.9700 - val_loss: 0.6931 - val_accuracy: 0.8390

```

Plotting the training results

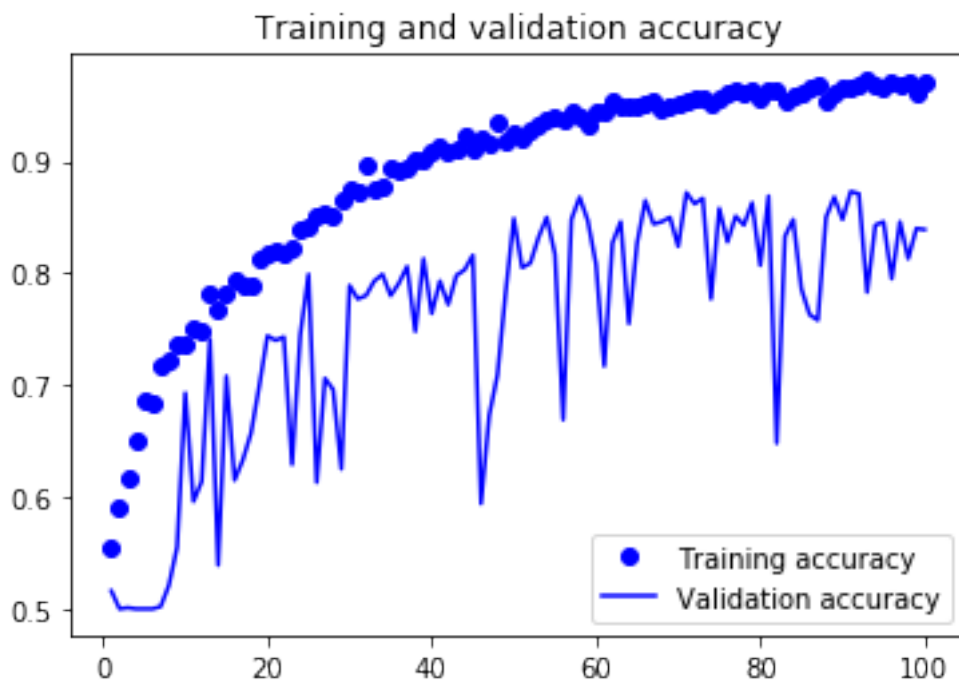
```
[8]: import matplotlib.pyplot as plt
```

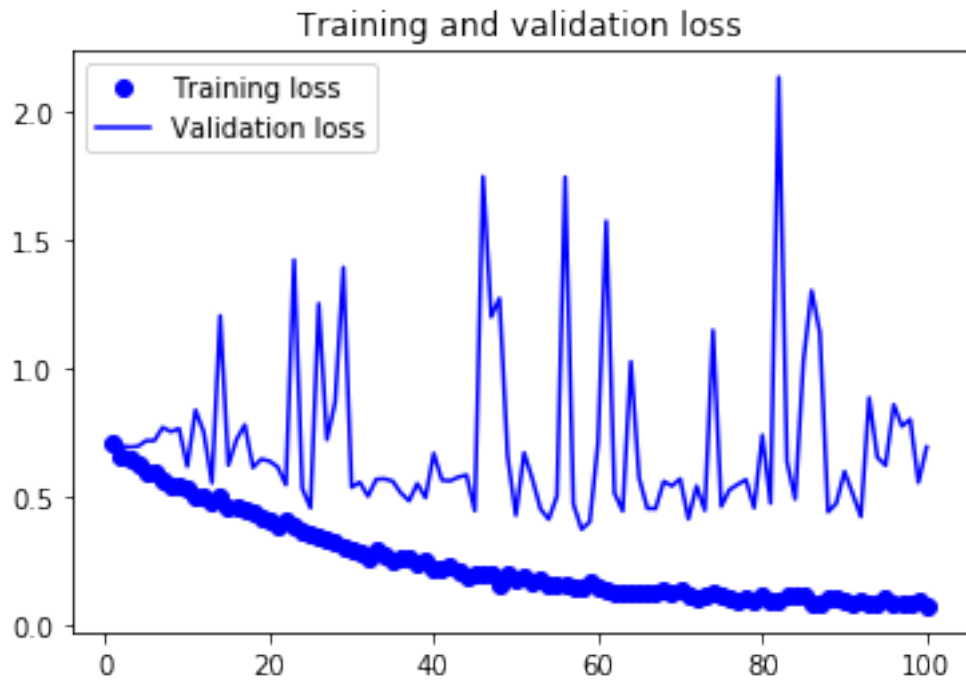


```

accuracy = history.history["accuracy"]
val_accuracy = history.history["val_accuracy"]
loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(accuracy) + 1)
plt.plot(epochs, accuracy, "bo", label="Training accuracy")
plt.plot(epochs, val_accuracy, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()

```





Evaluation on the test dataset

```
[9]: test_model = keras.models.load_model("mini_Xception_like.h5")
test_loss, test_acc = test_model.evaluate(test_dataset)
print(f"Test accuracy: {test_acc:.3f}")
```

```
63/63 [=====] - 2s 21ms/step - loss: 0.4064 - accuracy:
0.8610
Test accuracy: 0.861
```